NOM : _____

Contrôle Continu Programmation C++

26/11/2025 - Durée : 1h30 - Documents papiers autorisés

Attention : Vous veillerez à écrire du code le plus propre possible respectant les règles présentées en cours. Notamment, vous ferez attention à nommer explicitement vos variables, vos fonctions ou vos méthodes, à utiliser à bon escient les références, à utiliser correctement le mot clé « const » dès que nécessaire, à respecter les principes SOLID, …

Exercice 1 : QCM – Compréhension des concepts fondamentaux du C++ (7 points)

Répondez aux 14 questions en reportant vos réponses dans le tableau ci-dessous. Attention, les questions peuvent comporter plusieurs réponses possibles.

Pour chaque question, 0.5 point est attribué si l'ensemble des réponses cochées est entièrement correct. Si la réponse comporte une seule erreur (oubli d'une réponse juste ou ajout d'une réponse fausse), 0,25 point est attribué. Sinon, la question est notée 0.

	a	b	c	d
Q01				
Q02				
Q03				
Q04				
Q05				
Q06				
Q07				
Q08				
Q09				
Q10				
Q11				
Q12				
Q13				
Q14				

Q01 Quelles affirmations sont correctes à propos du mot-clé 'const' lorsqu'il est utilisé pour qualifier une méthode de classe en C++ ?

a) Une méthode marquée 'const' ne peut appeler que d'autres méthodes 'const' de la classe.
b) Il empêche la redéfinition de la méthode dans une classe dérivée.
c) Il garantit que la méthode ne modifie pas les attributs de l'objet.
d) Il transforme une variable en constante globale.

Q02 Quelles affirmations sont correctes à propos de l'encapsulation

a) Elle vise à protéger les données internes d'une classe.
b) Elle impose que tous les attributs soient publics.
c) Elle encourage l'utilisation d'attributs privés.
d) Elle garantit l'absence de bugs.

Q03 Parmi les relations suivantes, laquelle correspond à une relation de type 'is-a'?

a) Un compte épargne hérite d'un compte bancaire.
b) Une bibliothèque contient plusieurs livres.
c) Une voiture utilise un moteur thermique.
d) Un moteur consomme du carburant.

Q04 Quelles affirmations sont correctes à propos du passage de paramètres en C++ ?

a) Passer par référence évite la copie.
b) Passer par valeur peut être coûteux pour les objets lourds.
c) Une référence peut être nulle.
d) Une référence ne peut pas être réassignée après sa création.

Q05. Quelles affirmations sont correctes à propos de la séparation des fichiers .h et .cpp ?

a) Le fichier .h contient uniquement les déclarations (interfaces) des classes et des fonctions.
b) L'inclusion d'un fichier .h dans un fichier .cpp permet d'indiquer au compilateur que certaines classes et fonctions sont définies ailleurs.
c) On peut mettre toute l'implémentation dans le .h sans conséquence.
d) La séparation facilite la réutilisation et la compilation séparée.

Q06. Soit une classe TodoManager qui gère une liste de tâches (création, mise à jour, sauvegarde dans un fichier et affichage dans la console). Selon le principe SRP, quelles affirmations sont correctes ?

a) Cette classe a une seule responsabilité, celle de gérer une liste de tâches.
b) L'affichage devrait être géré en dehors de la classe.
c) Le stockage des taches dans un fichier appartient à la logique métier de gestion des tâches.
d) SRP impose une classe par méthode.

Q07. Quelles affirmations sont correctes à propos de la création d'une classe ?

a) Les attributs ne doivent pas être public.
b) La validation des données doit être faite dans le constructeur.
c) La classe doit garantir qu'un objet est toujours dans un état valide tout au long de sa vie.
d) Une méthode de la classe ne devrait pas modifier plusieurs attributs non liés.

Q08 Concernant le processus de compilation, quelles affirmations sont vraies ?

a) La compilation transforme le code source en un programme exécutable.
b) Elle permet de détecter certaines erreurs avant l'exécution.
c) Elle garantit l'absence de bugs à l'exécution.
d) Elle lit le code et exécute les instructions.

Q09 Pourquoi utilise-t-on des conventions de nommage ?

a) Pour permettre au compilateur d'optimiser.
b) Pour uniformiser le code et améliorer la lisibilité.
c) Pour rendre le code compatible avec d'autres langages.
d) Pour éviter les erreurs de compilation.

Q10 Quel est l'intérêt principal de créer des classes en programmation orientée objet ?

a) Réduire la taille du code source.
b) Organiser le code en entités cohérentes et réutilisables.
c) Accélérer automatiquement l'exécution du programme.
d) Éviter d'avoir à utiliser des fonctions.

Q11 Concernant les constructeurs en C++, quelles affirmations sont correctes ?

a) Un constructeur porte le même nom que la classe.
b) Un constructeur peut retourner une valeur.
c) Le constructeur sert à initialiser l'objet.
d) Une classe peut avoir plusieurs constructeurs.

Q12 Concernant la visibilité des membres d'une classe (public, private, protected), quelles affirmations sont correctes ?

a) Les attributs sont généralement en private.
b) Les méthodes destinées à l'utilisateur sont en général en public.
c) Les membres private sont accessibles dans les classes dérivées.
d) Les membres protected sont accessibles dans les classes dérivées.

Q13 Concernant les constructeurs et l'héritage en C++, quelles affirmations sont correctes ?

a) Le constructeur de la classe mère est appelé avant celui de la classe dérivée.
b) Le constructeur de la classe dérivée initialise directement les attributs de la classe mère.
c) On peut initialiser les attributs de la classe mère dans le code du constructeur de la classe dérivée.
d) Il est préférable d'initialiser les attributs via la liste d'initialisation du constructeur.

Q14 Parmi les pratiques suivantes, lesquelles vont dans le sens du principe OCP ?

a) Une classe doit être conçue de manière à pouvoir être étendue.
b) Une classe doit contenir le plus de fonctionnalités possibles.
c) L'ajout de fonctionnalités doit se faire sans modifier le code existant.
d) Une classe doit être réécrite dès qu'un nouveau besoin apparaît.

Exercice 2 : Modélisation d'un stationnement (8 points)

On modélise un système simple de gestion de stationnement. On distingue une session de stationnement standard `ParkingSession` et une session de stationnement payante `PaidParkingSession`.

Q01 Créer une classe `ParkingSession` avec les attributs : immatriculation (`string`), durée autorisée de stationnement en minutes (`int`) et état indiquant si la session est active (`bool`). On impose que la plaque d'immatriculation ne soit pas vide, que la durée soit strictement positive et que la session soit toujours créée active. Écrire la déclaration de la classe.

Q02 Écrire le code du constructeur et des getters.

Q03 Écrire le code de la méthode `isExpired(int elapsedMinutes)` qui renvoie `true` si le temps écoulé, strictement positif (`elapsedMinutes`) est supérieur à la durée autorisée du stationnement.

Q04 Écrire le code de la méthode `extend(int additionalMinutes)` qui prolonge la durée autorisée si `additionalMinutes` est strictement positif ;

Q05 On souhaite maintenant gérer des sessions de stationnement payantes. Une session payante `PaidParkingSession` est une session `ParkingSession` à laquelle on ajoute un attribut représentant le tarif horaire en euros/heure (`double`). On impose que le tarif horaire soit strictement positif avec une valeur par défaut de 5 euros. Écrire la déclaration de la classe.

Q06 Écrire le code du constructeur et le code du getter.

Q07. Écrire le code de la méthode `computeCost()` qui renverra le coût total du stationnement en utilisant la durée autorisée et le cout horaire.

Q08. Écrire un court extrait de code qui crée un objet `PaidParkingSession` pour une plaque `"AB-123-CD"`, une durée de 90 min et un tarif de 2.5 €/heure, puis qui affiche le coût total du stationnement.

Exercice 3 : Analyse et correction d'une classe Order (5 points)

On considère la classe suivante `Order`, qui modélise (très simplement) une commande passée par un client dans une boutique en ligne. Pour des raisons de concision, le code fourni ne présente pas le code des getters/setters des 5 attributs, ni le code de la classe `Customer` qui représente un client.

Fichier order.h

```
class Order {
public:
    Customer customer_;
    std::string product_;
    int quantity_;
    double price_;
    bool paid_;

    Order(Customer customer, std::string product,
          int quantity, double price);
    double totalPrice();
    void display() ;
};
bool pay(Order order, double amount);
```

Fichier order.cpp

```
Order::Order(Customer customer, std::string product,
             int quantity, double price) {
    customer_ = customer;
    product_ = product;
    quantity_ = quantity;
    price_ = price;
    paid_ = false;
}
double Order::totalPrice() {
    return quantity_ * price_;
}
void Order::display() {
    std::cout << customer_ << " ordered "
              << quantity_ << " x " << product_ << std::endl;
    }
bool pay(Order order, double amount) {
    if (amount == order.totalPrice()) {
        order.paid_ = true;
        return true;
    }
    return false;
}
```

Pour chacune des 4 questions suivantes, vous devrez analyser le code et décrire les problèmes de conception que vous observerez. Dans la 5ème question, vous proposerez un code « propre » qui respecte les règles fondamentales de Programmation Orientée Objet en C++ présentées en cours.

Q1 Analyser le fichier order.h.

Q2 Analyser le constructeur écrit dans le fichier order.cpp.

Q3. Analyser les deux méthodes `totalPrice()` et `print()`.

Q4. Analyser la fonction `pay(Order order, double amount)`.

Q5. A partir de votre analyse, réécrire les fichiers order.h et order.cpp.